

Reminder and Delayed Notification Workflow

1. Overview & Implementation Options Comparison

Sending timely show reminders (e.g., 2 hours before showtime) requires reliable delayed task execution at scale. We evaluate three architectural patterns:

Approach	Architecture	Scalability & Performance	Verdict
Scheduled DB Polling (Cron / Quartz)	Runs periodic SQL queries (SELECT ... WHERE show_time = NOW() + 2h) against PostgreSQL.	Poor: Causes high database CPU utilization, row-level read locks, and table scans as bookings grow into millions.	Rejected
Message Broker Delay Topics	Employs multi-bucket delayed Kafka topics or external broker delay plugins.	Moderate: High broker complexity, difficult cancellation handling, and disk write amplification.	Rejected
Distributed Delay Queue (Redis ZSET)	Bookings are stored with target reminder timestamps as scores; workers atomically fetch due jobs.	Excellent: In-memory sub-millisecond lookups, O(log N) scheduling, and instant job cancellation via key removal.	Selected

2. Scheduling & Polling Algorithms

When a customer completes a booking, ReminderService schedules the notification job in the delayed queue. Independent scheduler workers periodically fetch and process due reminders:

```
Algorithm: ScheduleReminder(event)
1. reminder_time = event.data.showTime - 2 hours
2. If reminder_time > CurrentTime():
    Add bookingId to delayed queue with trigger time = reminder_time

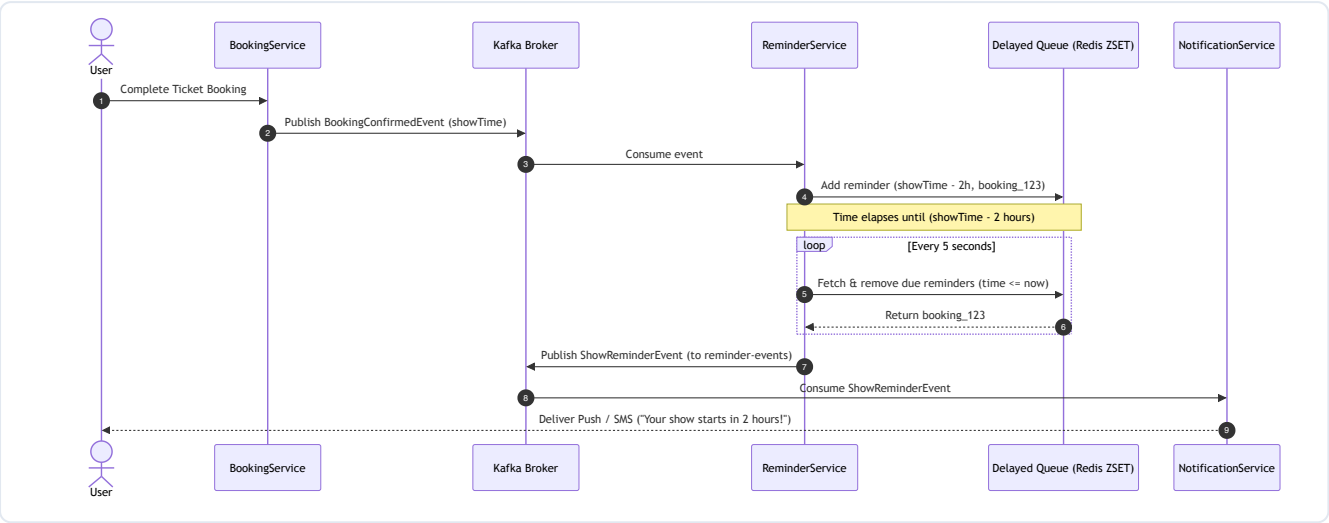
Algorithm: ProcessDueReminders()
1. Fetch all reminders where trigger time <= CurrentTime()
2. Atomically remove fetched reminders from the delayed queue
3. For each reminder:
    Publish ShowReminderEvent to Kafka topic "reminder-events"
```

3. Reliability & Duplicate Prevention

- **Handling Booking Cancellations:** If a customer cancels a ticket, BookingService publishes BookingCancelledEvent. ReminderService consumes this event and immediately deletes the corresponding bookingId from the delayed queue, ensuring no reminder is sent for cancelled tickets.
- **Atomic Dequeue:** When multiple scheduler workers run concurrently, fetching and removing due reminders occurs atomically. This guarantees that a single reminder job is never picked up by two workers simultaneously.
- **Consumer Deduplication:** The emitted ShowReminderEvent carries a deterministic identifier (reminder:BKG00123:2h). NotificationService verifies this ID against its processed event cache before dispatching any SMS or push alert.

4. Sequence Diagram: End-to-End Reminder Lifecycle

The sequence diagram below traces the entire lifecycle from initial ticket purchase to scheduled reminder delivery:



5. Dynamic Reminder Windows & Timezone Handling

- **Tiered Notification Windows:** The system supports multiple reminder tiers per booking: a 24-hour advance preparation email and a 2-hour immediate mobile push notification.
- **UTC Standardization:** All showtimes and reminder timestamps are stored and scheduled strictly in UTC epoch seconds, eliminating timezone translation bugs across regional cinema theaters.